SysBuilders — Bug Hunt & SOLID / Clean-Code Review

Scope: every script under `Assets/Scripts` (Core, Managers, Network, UI), `requirements.json`, the asmdefs, and the test inventory, cross-checked against and . Third-party Mirror code was excluded.

Method: static read-through only. I did **not** run the EditMode tests or open the game (Unity `6000.3.21f1` is installed, so I can run the tests if you want). Items marked *likely* need confirmation in the Editor. All scripts are valid UTF-8, so there is no encoding problem.

1. Bugs

🔴 High

#	Problem	Where	Why it's a bug
B1	**One "Criar Classe" click creates two classes and burns the whole turn quota (2/2).**	and	`HandleCreateClassClicked` creates a class and calls `NotifyClassCreated()`. It then fires `OnCreateClassClicked`, and `GameManager.HandleCreateClassRequested` is subscribed to that event. It passes `CanCreateClass()` (1 < 2) and creates a **second** class. Two code paths do the same job. A related naming issue: the toolbox names classes `Classe_{createdThisTurn+1}`, and that counter resets every turn. On turn 2 the name `Classe_1` collides with turn 1 and the add fails silently, so GameManager's fallback creates the class instead.
B2	**Refactor is unreachable: nobody ever awards Refactor Points.**		`AddRefactorPoints` has no caller in `Assets/Scripts`, only in tests. Points stay at 0, `RightBar` keeps the button disabled (`points >= cost`), and `CanRefactor` is always false. The GDD's "Custo de Refatoração" mechanic can't be played. Neither doc says how points are earned, so this needs a **design decision** first.
B3	**`MatchSession.RemoveClass` always returns `false` and never raises `OnDiagramChanged`.**		`LocalPlayer.RemoveClass(node);` is called, then `if (!LocalPlayer.RemoveClass(node)) return false;` calls it again. The second call fails because the class is already gone. The data is removed but the UI is never told. It is latent today (no UI button calls it, and `DiagramController.TryRemoveClass` is unused), and no test covers the `MatchSession` path.

🟠 Medium

#	Problem	Where	Notes
B4	The same `MethodData` can be added many times to one class.	,	Only `GameManager.HandleMethodCardSelected` has a `Contains` guard, and it runs *after* the toolbox already added the card. Repeated clicks inflate `Methods.Count`, which triggers false **Large Class** penalties. The invariant belongs in the domain, not in one UI subscriber.
B5	Unplaced methods from earlier turns disappear, but earlier requirements keep being scored.		The toolbox shows only `CurrentRequirement` methods. "Regra B" re-verifies *all* revealed requirements each turn. A method you forgot on turn 2 can never be placed later, so those 100 points are lost for good. Confirm whether that is intended.
B6	*Likely:* the Editor auto-start can pre-empt the networked start.	vs	`Start()` runs on the first frame. A Mirror client only gets `_trailId` after a round-trip. In the Editor with `_autoStartInEditor = true`, a client can start an *offline PlayerOne* match first. The real `StartMatch` is then ignored with a warning, and `RequireOpponentResult` is set after the fact (`requiredPlayers = 1`). Builds are unaffected.
B7	`RpcAdvancePhase` throws a `NullReferenceException` if no match exists.		`gameManager.Session.TurnManager` has no null guard, unlike `RpcReceiveEvaluationResult`. It can happen after an aborted start or a late RPC.
B8	*Likely:* the Editor offline mode never shows the turn-end modal.		The session is built with `requiredPlayers = 1`, but the harness then calls `SetPlayerReady(PlayerTwo)` while the phase is `Show`. That alone satisfies the tracker, so `Show → Build` fires in the same click chain that opened the modal.

🟡 Low

#	Problem	Where
B9	`MenuSceneManager.StartGame` loads `GameScene` offline. Outside the Editor nothing calls `StartMatch`, so the player gets a dead scene. It also duplicates `MenuManager`'s panel code and isn't in the roadmap.	
B10	The server trusts client-reported `rawScore`, `penaltyScore` and `turnNumber`, and has no phase awareness (`_readySlots` and `_evaluatedSlots` are phase-agnostic). `GameRoomManager` accepts *any* server public key, so the encryption does not authenticate the host. This is acceptable for a classroom LAN, but cheating is possible.	,
B11	Match score can go negative (`CalculateScore`), while `EvaluationResult.TotalScore` clamps at 0. Two rules for "score".	,
B12	Event/lifecycle leaks: `GameManager.UnwireUI` is never called, and `OnDestroy` exists only under `#if UNITY_EDITOR`. `MatchSession.Dispose()` exists but the class isn't `IDisposable` and nothing calls it.	
B13	`MatchSession.AddMethodToClass` and `AddRelation` don't check that the nodes belong to the local diagram. `AddClassToPlayer` is unused, bypasses `OnDiagramChanged`, and lets callers pick any slot.	
B14	Mixed line endings in `Relation.cs`, `TopMenuController.cs` and `TopMenuControllerTests.cs`. These create noisy diffs.	—

2. GDD / Roadmap vs. code

Doc says	Code does
GDD: classes get **free names** chosen by the player.	Names are auto-generated (`NextClassName` has a TODO). The toolbox has an input field, but see B1.
GDD lists 3 smells (Large Class, Duplicate Code, Baixa Coesão).	A 4th exists: `LowAffinityCoupling` . **Update the GDD** or confirm it is in scope.
Roadmap `ITurnContext` has `RemainingTurnTime` , `OnTimerTick` and `OnPhaseChanged` .	The code has `OnPhaseChange` and no timer. A separate `ITurnTimerContext` exists but is never wired (the GDD marks the timer as an open hypothesis, so fine).
Roadmap `IRefactorContext.Available RefactorPoints` .	`RefactorPoints` , plus extra `…Selected` members.
Roadmap: `GameManager` has **zero network code**.	`RequireOpponentResult` , `ReceiveOpponentEvaluation` and `OnReturnToMenuRequested` leak network concepts (mild, but a mutable flag that must be set *before* `StartMatch`).
Roadmap: `MenuManager` lives in the Managers layer and `RequirementManager` exposes 2 methods.	`MenuManager` is in `Network/` . `MenuSceneManager` is undocumented. `RequirementManager` exposes ~8 members, several unused.
Roadmap: Core is Unity-free.	True in the code, but `Sys.Core.asmdef` has `noEngineReferences: false` , so nothing enforces it.
✅ Matches	Association-only with a `RelationType` enum, no attributes, 5-turn trails, Show→Build→Evaluate→Completed, primitives-only over Mirror, `localSlot` injection, ISP context interfaces.

3. SOLID assessment

Overall: a solid foundation, with a few god-objects and some leaky seams.

- **S — Single Responsibility (⚠️ weakest)**
  - (~350 lines) mixes turn orchestration, scoring math, diagram commands, refactor, error messages, opponent results, and **UI state** (`SelectedClass` , `IsAssociationMode`). It implements 6 interfaces.
  - is the composition root, but also generates class names, handles method-card clicks, runs the advance coroutine, tracks evaluation flags, does scene navigation, and contains a full

Editor test harness.

  - `ToolboxController` enforces a *domain rule* (max classes per turn) that `PlayerState` only mentions in a TODO.
  - `TurnManager.cs` holds 5 types (`PlayerSlot` , `TurnPhase` , `SetPlayerReadyResult` , `PlayerReadinessTracker` , `ITurnPlayerActions`).

- **O — Open/Closed (⚠️)**
  - Adding a smell (Phase 2: Lazy Class, Over-engineering) means editing `CodeSmellType` , `CodeSmellDetector` , `SmellPenaltyCalculator` , and `EndGameModalController.TranslateSmell` . An `ICodeSmellRule` list, each rule carrying its own penalty and label, would make Phase 2 additive.
- **L — Liskov (✅ n/a)**
  - There is almost no inheritance in the domain, consistent with the GDD. One naming trap: `RequirementManager.GetTrailById` returns `IReadOnlyList<Requirement>` , while the explicit `IRequirementProvider.GetTrailById` returns `Trail` .
- **I — Interface Segregation (✅ mostly)**
  - The `I…Context` interfaces are the best part of the design. `IDiagramActions` is the fat one: 13 members covering commands, selection, mode and 4 events.
- **D — Dependency Inversion (✅ / ⚠️)**
  - Good: `IEvaluationService` injection, UI depends only on interfaces, Core knows nothing about Unity or Mirror.
  - Violations:
    - `MatchSession` does `new TurnManager / PlayerState / RefactorHandler` and takes a `bool isNetworkMatch` .
    - `EvaluationEngine` news its own collaborators.
    - `GameManager` depends on 6 concrete controllers.
    - `MatchSyncBridge` reaches through `gameManager.Session.TurnManager.ForceAdvancePhase()` (Law of Demeter), and `ForceAdvancePhase` is public to everyone.
    - `Sys.Network.asmdef` references `Mirror.Editor` , `Mirror.Examples` , `Mirror.Authenticators` and `Mirror.CompilerSymbols` . The editor-only reference is a **build risk** (verify with a Player build). Trim the references to what is used.

4. Clean-code findings

- **Duplicated paths for one action:** class creation (B1), method-card add (Toolbox + GameManager), association mode kept in three places (Session, Diagram, Toolbox) and synced by events, and refactor (RightBar calls `TryRefactorSelectedClass`, then GameManager re-checks and re-renders).
- **Boilerplate repeated 6×:** `SetupButtons` / `UnsubscribeFromEvents` / `OnDestroy` in every controller. `Show/HideModal` is duplicated in two modals. The `Destroy` vs `DestroyImmediate` helper appears 3×.
- **Magic values:** `"SelectedTheme"` and `"PlayerName"` (4 files), `"GameScene"`, `"Menu"`, `"Biblioteca"`, spawn offsets `160/40/600`, `2.5f`, `MaxSmellsPerResult = 50`, `_maxClassesPerTurn = 2`.
- **Dead code:** `RequirementManager` (`Catalog`, `CurrentTheme/TrailId/Requirements`, `SetActiveTrail`, `SelectTrail`, `GetRequirementsForMatch`), `GameRoomManager.ChosenTrailId`, `GameManager.Initialize` and `_opponentEvaluated`, `ITurnTimerContext`, `AddClassToPlayer`, `DiagramController.TryRemoveClass`.
- **Naming and language:** `OnPhaseChange` vs `OnTimeChanged`, mixed PT/EN identifiers (`AbrirStart` vs `StartGame`), stray thinking-aloud comments (e.g. in `TurnManager`).
- **Test gaps:** nothing covers the `GameManager` + `ToolboxController` interaction (B1), `MatchSession.RemoveClass`, `RequirementCatalog`, or the bridge's RPC flow (B7). The existing ~2,400 lines of tests are strong on Core scoring and evaluation.

5. What's done well

Pure-C# Core, DTO validation with clear errors in `RequirementLoader`, defensive copies in `EvaluationResult` and `SmellOccurrence`, ordered/duplicate-safe `ReceiveOpponentResult`, slot-authorization checks on every Mirror command, an O(V+E) connected-components pass, and a table-driven penalty calculator that fails loudly on unmapped smells.

6. Suggested fix order

1. **B1** — one owner for class creation (remove the GameManager path, or the toolbox's own creation).
2. **B3** — delete the duplicate `RemoveClass` call, and add a test.
3. **B4** — reject duplicate `MethodData` inside `ClassNode` / `MatchSession`.
4. **B7** and **B6** — null-guard the RPC, and gate the Editor auto-start (move it into a separate `EditorOfflineBootstrap`).

5. **B2** and **B5** — decide the rules (how refactor points are earned, whether old methods stay available), then implement and document them.
6. Architecture pass: extract selection/mode state out of `MatchSession`, move the max-classes rule into the domain, introduce `ICodeSmellRule`, set `noEngineReferences: true` on `Sys.Core`, and prune the `Sys.Network` asmdef references.
7. Sync and the GDD with the divergences in §2.